

LUPA/1: transmisión de teselas gobernada por intención y recursos

Especificación de la implementación - RFC académico

Identificación	Valor
Documento	A24 / RFC-LUPA-1, revisión 3
Fecha	6 de octubre de 2026
Consolidación	Adrian Romero
Estado	Especificación del código existente; no es un RFC publicado por IETF
Subprotocolo / versión	lupa.v1 / 1
Código de referencia	f57fb91465ad0f9d1e9d783d0deb5e9453899bd8
Repositorio	bitvalo34/prj_LUPA

Resumen

LUPA es un protocolo de aplicación para consultar regiones de imágenes cuya representación completa no debe descargarse ni decodificarse en el navegador. El cliente comunica su intención de observación; el servidor selecciona teselas JPEG, prioriza contexto y foco, cancela trabajo superado y limita las entregas pendientes mediante créditos. El receptor confirma el consumo terminal de cada entrega y puede solicitar recuperación selectiva. La admisión de lecturas entre sesiones utiliza una variante de Deficit Round Robin (DRR).

Las pirámides constituyen el almacenamiento previo. El protocolo coordina selección, progreso, cancelación, identidad, contrapresión, recuperación y liberación de recursos. Este documento describe el comportamiento implementado y presenta su arquitectura, procesamiento de imágenes y resultados experimentales.

Revisión 3: consolidación final de Erwin Arevalo y Adrian Romero. Incluye arquitectura, ingestión, capturas y mediciones Canvas (pp. 22-27). El candidato es la base indicada más la actualización local del asset servido por Java; sus hashes acompañan la evidencia.

Contenido

1. Alcance y convenciones	3
2. Transporte y apertura	5
3. Estados, identidad y orden	6
4. Controles JSON	7
5. Estructura binaria TILE	10
6. Selección y progreso de la vista	10
7. Crédito y cancelación	12
8. STA: recuperación selectiva de entregas	13
9. Recursos, caché y equidad	14
10. Fallos, plazos y reconexión	16
11. Validación, resultados y límites de las afirmaciones	17
12. Trazabilidad al código y referencias	20
Apéndice A. Intercambio concreto de ejemplo	21
13. Versión de implementación y validación	22
14. Arquitectura implementada	23
15. Procesamiento y almacenamiento	24
16. Campaña visual A24: método y resultados	25
17. Evidencia del visor y liberación	26
18. Foco, importación y protección	27

Las secciones 2 a 10 especifican el intercambio y sus controles. La sección 11 distingue evidencia medida y metas; la sección 12 vincula los mecanismos con el código mediante las claves C01-C26. Las secciones 13 a 18 presentan la versión validada, la arquitectura, el procesamiento y los resultados experimentales.

1. Alcance y convenciones

Esta especificación cubre la negociación, mensajes, máquina de estados, algoritmos y controles LUPA. HTTP y WebSocket sostienen el intercambio. Las secciones 14-18 completan la arquitectura, la ingestión, la evidencia del visor y la reproducción de la implementación.

MUST, MUST NOT, SHOULD y MAY indican obligación, prohibición, recomendación y posibilidad dentro de este perfil académico, conforme a la convención de [RFC 2119](#) y [RFC 8174](#). No implican certificación externa. Las constantes y valores de configuración son los del código fijado arriba; no representan capacidades universales.

Término	Significado operativo
Época (epoch)	Número creciente de intención aceptada en una conexión: apertura o vista.
Entrega (deliveryId)	Identidad de una tesela enviada; se mantiene al retransmitirla.
Versión de imagen	Identidad inmutable abierta por OPEN, distinta de la versión del protocolo.
Comprometido	La cola de escritura seleccionó el mensaje para escribirlo; ya no es cancelable. No significa recibido.
Escrito	Terminó la escritura local completa del mensaje al socket. No significa decodificado.
Terminal	El cliente dibujó, descartó o abandonó definitivamente la entrega.
Presentado	Se ejecutó drawImage y su callback local. No prueba percepción humana ni finalización física del monitor.
Crédito	Bytes de envolturas LUPA originales pendientes de consumo terminal; no es la ventana TCP.

deliveryId y epoch son enteros entre 1 y 2147483647; imageId e imageVersion son cadenas. El servidor no reutiliza deliveryId dentro de una conexión. Agotar la secuencia exige una sesión nueva. connectionId y planGeneration son guardas internas; MUST NOT confundirse con campos transmitidos.

1.1 Decisiones de diseño y compromisos

Los motivos siguientes describen efectos técnicos verificables en el código. No presuponen una comparación de rendimiento contra todos los diseños alternativos ni afirman que los parámetros elegidos sean óptimos.

Problema y decisión	Motivo técnico	Costo o límite
Solicitudes frecuentes: un WebSocket persistente	Permite controles cliente-servidor y teselas servidor-cliente en el mismo canal, sin GET por tesela. [C02, C13]	Comparte el orden de TCP; no ofrece streams independientes ni elimina su bloqueo por bytes anteriores.
Control legible y datos voluminosos: JSON + JPEG binario	JSON permite inspeccionar intención y respuesta; el JPEG no se convierte a Base64. H delimita inequívocamente la cabecera dentro del mensaje binario. [C06, C09]	Hay costo de cabecera y parseo; JPEG no garantiza fidelidad sin pérdida.

Problema y decisión	Motivo técnico	Costo o límite
Pan/zoom superpuestos: época creciente y generación interna	La comparación de época descarta intenciones superadas; la generación invalida callbacks normales que aún estaban trabajando. [C05, C10]	No retira bytes comprometidos ni interrumpe una lectura ya en ejecución.
Receptor más lento que el servidor: créditos de consumo y máximo de entregas	El límite en bytes considera JPEG de tamaños distintos; el límite de 16 entregas también acota metadatos y trabajos pendientes. Confirmar después del consumo protege el pipeline, no solo el socket. [C07, C09]	Un cliente que no confirma se pausa y puede expirar; la ventana no se ajusta dinámicamente.
No dejar la región sin atención: contexto antes del detalle	La base z0 y el contexto del foco aportan una representación temprana; después llegan las teselas que refinan la intención vigente. [C08, C17]	El contexto agrega bytes y puede verse menos definido durante el refinamiento.
Fallo de una entrega: recuperación selectiva con el mismo id	Releer solo la referencia fallida evita repetir el plan completo y conserva una única reserva de consumo. [C11, C15]	Dos reintentos como máximo; las relecturas selectivas no pasan por DRR.
Sesiones con JPEG de costos distintos: variante DRR	El costo comprimido observado influye en la siguiente elegibilidad, en lugar de tratar cada tesela como una unidad de costo idéntico. [C12]	Es una estimación acotada del próximo costo; no garantiza latencias iguales ni equidad exacta del tráfico de red.
Reutilización sin mezclar imágenes: versión fijada y LRU compartida	La clave incluye versión y coordenadas; una tesela publicada inmutable se puede compartir sin confundir datos de v1 y v2. [C05, C20]	Misses concurrentes pueden duplicar lecturas; no existe inventario de teselas cliente que suprima todos los reenvíos.

1.2 Restricciones, parámetros y resultados

Las restricciones del perfil, la negociación y la configuración de recursos no son equivalentes:

Categoría	Ejemplos y efecto
Contrato del perfil	Versión 1, codec JPEG, tesela de 256, controles ≤ 16 KiB, cabecera TILE ≤ 4096 y payload ≤ 262144 . Cambiarlos exige coordinar la compatibilidad del servidor y del cliente. [C06, C14]
Negociación por conexión	HELLO anuncia ventana y presupuesto bitmap; el servidor limita la ventana al rango admitido. El navegador actual anuncia 64 MiB, pero ese valor no es un máximo universal del campo HELLO. [C04, C15]
Configuración de ejecución	32 sesiones, 8 lecturas normales admitidas, caché JPEG de 128 MiB, buffers transitorios de 16 MiB, quantum y plazos. Son parámetros base sujetos a validaciones, no máximos de hardware. [C19]

E24 aporta evidencia para conservar la configuración base en la carga medida, no una demostración de optimalidad. La tabla de recursos de la sección 9 y los resultados de la sección 11 mantienen separadas esas dos afirmaciones.

2. Transporte y apertura

2.1 Separación de responsabilidades

Capa	Responsabilidad
TCP del sistema operativo	Flujo fiable y ordenado de bytes, retransmisión de segmentos y control de congestión.
WebSocket	Apertura HTTP, límites de mensajes, texto/binario, máscara cliente, Ping/Pong y Close.
LUPA	Intención vigente, selección de teselas, crédito de consumo, confirmación selectiva y recursos del visor.

La referencia TCP es [RFC 9293](#), [sección 3.8.6](#). LUPA no implementa TCP, no modifica su ACK y no agrega control de congestión propio. Una entrega declarada fallida por LUPA puede haber llegado correctamente por TCP.

2.2 HTTP inicial y WebSocket

El servidor Java entrega GET /, recursos locales /assets/... y GET /api/catalog. El catálogo usa Cache-Control: no-store; las rutas públicas no exponen originales ni permiten recorrido ... El catálogo identifica imágenes, pero no sustituye el manifiesto de una apertura. [C01]

El navegador abre /lupa en el mismo host, con Sec-WebSocket-Protocol: lupa.v1. La apertura exige GET, versión WebSocket 13, Upgrade/Connection válidos y una clave Base64 que decodifique 16 bytes. La respuesta exitosa es HTTP 101 con el subprotocolo seleccionado. No se negocian extensiones de compresión. [C02]

El servidor procesa tramas RFC 6455 incrementalmente, exige máscara en las del cliente, rechaza RSV no negociados y longitudes no mínimas. Permite fragmentación de texto con reensamblado acotado y tramas de control intercaladas. El cliente MUST NOT enviar mensajes binarios de aplicación: se cierran con 1003. LUPA envía controles en texto UTF-8 y cada TILE como mensaje binario. [C03; RFC 6455, secciones 4, 5 y 7]

La política de origin admite orígenes locales o el host/puerto solicitado según su validador. Por defecto permite ausencia de origin para clientes técnicos. No equivale a autenticación ni a una política universal de origen estricto. El servidor actual no termina TLS; wss requiere una terminación externa si se despliega fuera del laboratorio.

3. Estados, identidad y orden

3.1 Máquina de estados del servidor



Cualquier estado -- cierre o error fatal --> CERRADA

ACK_STATE y RELEASE se admiten después de HELLO, incluso si confirman entregas de épocas anteriores. Un OPEN válido cambia la imagen abierta; un VIEW válido reemplaza el plan vigente. Los fallos recuperables no equivalen a una transición exitosa. [C04]

3.2 Regla de última intención aceptada

OPEN y VIEW usan la misma secuencia creciente. El servidor ignora una intención cuyo epoch $\leq$ currentEpoch; una intención inválida no adelanta la época ni destruye el plan válido anterior. La resolución asíncrona de OPEN vuelve a comprobar la época antes de instalar la imagen. Un VIEW MUST identificar exactamente la imagen y versión abiertas. [C05]

OPEN fija una versión. Si el catálogo publica v2 mientras la conexión observa v1, la sesión no mezcla versiones ni migra automáticamente: otro OPEN toma la versión actual. Los niveles de MANIFEST van de z0, miniatura de una tesela, al nivel nativo; el cliente MUST validar geometría e identidad antes de dibujar. Una ampliación digital sobre el nivel nativo no agrega información a la imagen. [C05, C14]

3.3 Precondiciones y efectos de los controles

Control recibido	Precondición / resultado normal	Rechazo o conservación de estado
HELLO	Solo ESPERA_HELLO. Con negociación y admisión válidas: LISTA + WELCOME.	Versión positiva no soportada o presupuesto insuficiente: ERROR, sin negociación. Repetir HELLO tras WELCOME: cierre 1008.
OPEN	LISTA o IMAGEN_ABIERTA. Solo al resolver metadatos válidos de una época mayor instala imagen/versión y emite MANIFEST.	Antes de HELLO: 1008. Imagen ausente/no disponible: ERROR; conserva imagen/plan anterior y no consume la época.
VIEW	IMAGEN_ABIERTA, identidad coincidente y parámetros/plan válidos. Reemplaza el plan, aumenta época/generación y emite PLAN.	Antes de OPEN exitoso: 1008. Vista inválida: BAD_VIEW; plan que no cabe: LIMIT_EXCEEDED. No sustituyen la intención aceptada.
ACK_STATE	Después de HELLO. Valida todos los rangos antes de liberar reservas de su época o procesar missing.	Antes de HELLO o estructura inválida: 1008. Id desconocido/ya confirmado no crea crédito; missing tiene las guardas de la sección 8.
RELEASE	Después de HELLO, id entero y status permitido. Devuelve una vez la reserva conocida, incluso de una época anterior.	Status inválido: 1008; id desconocido/duplicado: sin efecto.
Tipo desconocido o control JSON ilegible	No constituye una operación LUPA válida.	Cierre 1008, no una transición exitosa ni un DONE.

La regla de ignorar épocas antiguas de OPEN/VIEW se aplica después de las comprobaciones iniciales de tipo/estado indicadas en el código; no autoriza texto JSON ilegible. La apertura asíncrona no tiene un estado OPENING adicional en el servidor: conserva su estado previo hasta aceptar el resultado. Los estados visuales del navegador son distintos de esta máquina. [C04-C07]

En particular, un epoch faltante, no entero o fuera de rango en OPEN/VIEW falla en la validación inicial y provoca 1008. BAD_VIEW corresponde a la validación posterior de la vista, por ejemplo identidad, rectángulo, viewport, detalle o foco incorrectos. Esta distinción permite probar errores recuperables sin confundirlos con violaciones de formato/secuencia.

4. Controles JSON

4.1 Reglas comunes

En controles cliente-servidor, todos los campos listados son obligatorios, incluidos focus, received y missing. Al validar un control vigente, el servidor rechaza campos desconocidos y valores de tipo incorrecto; una intención antigua puede ignorarse antes de validar todos sus campos. El parser rechaza claves duplicadas y contenido posterior al objeto JSON. Los enteros no admiten fracciones. Límite del mensaje: 16384 bytes; profundidad: 8; campos por objeto: 32; elementos por arreglo general: 64; cadenas: 256 caracteres; nombres de campo: 64. ACK_STATE tiene límites menores adicionales. [C06]

imageId cumple `^[a-z0-9](?:[a-z0-9-]{0,62}[a-z0-9])?$` (máximo 64 caracteres); imageVersion cumple `^[1-9][0-9]{0,9}$`. El formato canónico de mode es minúsculo; el servidor actual también acepta variantes de mayúsculas.

4.2 Negociación y apertura

Mensaje / dirección	Campos, tipos y restricciones
HELLO / cliente -> servidor	type="HELLO"; version: entero, debe ser 1; windowBytes: entero positivo hasta 2147483647, mínimo aceptado 524288; bitmapBudgetBytes: entero positivo hasta el máximo long de Java, mínimo 16777216.
WELCOME / servidor -> cliente	type="WELCOME"; version=1; windowBytes: ventana aceptada, limitada a 1048576; maxTileBytes=262144; maxInFlight=16.
OPEN / cliente -> servidor	type="OPEN"; epoch: entero positivo creciente; imageId: identificador del catálogo. No contiene versión.
MANIFEST / servidor -> cliente	type="MANIFEST"; epoch: del OPEN; imageId, imageVersion: cadenas; width, height: dimensiones originales positivas; tileSize=256; levels: arreglo ordenado de {z,width,height}, con índices contiguos desde 0 y dimensiones positivas consistentes.

El navegador actual solicita 1048576 bytes de ventana y 67108864 bytes de presupuesto bitmap. WELCOME no devuelve un campo bitmapBudgetBytes; el servidor conserva el anunciado para planificar. No hay actualización dinámica de ventana. Si HELLO solicita menos que el mínimo se rechaza; si supera el máximo de ventana permitido, se limita. [C04]

Este perfil incorpora ACK_STATE manteniendo versión 1, sin negociar capacidades adicionales. El navegador no hace fallback automático a RELEASE; necesita el servidor de este perfil. Conservar RELEASE permite clientes anteriores contra el servidor actual, no garantiza que el cliente nuevo funcione con un servidor antiguo que desconozca ACK_STATE. [C04, C07]

4.3 Intención de vista y respuesta

Campo de VIEW	Tipo y semántica
type	Cadena literal VIEW.
epoch	Entero creciente, 1..2147483647.
imageId, imageVersion	Cadenas; deben coincidir con el OPEN exitoso.
rect	{x,y,width,height} enteros en coordenadas de la imagen original; x/y >= 0, width/height >= 1; rectángulo completamente dentro de la imagen. Intervalos semiabiertos.
viewportPx	{width,height} enteros positivos en píxeles físicos; área <= 8294400. El cliente incorpora DPR al preparar la vista.
detailOffset	Entero -2, -1 o 0: desplaza el nivel automático hacia menor detalle.
mode	uniform o focus.
focus	null en uniform; en focus, {x,y,radiusPx} enteros: punto dentro de la imagen y radio de 1..512 píxeles de viewport. Puede quedar fuera del rectángulo visible.

Respuesta	Campos y significado
PLAN	type, epoch, appliedLevel, contextLevel; los dos niveles son índices del manifiesto. No incluye lista de teselas ni promete una cantidad.
DONE	type, epoch, sentTiles; cantidad de envíos originales del plan escritos completamente. No cuenta retransmisiones ni confirma Canvas.
ERROR	type, code, message; epoch opcional si existe intención asociada. Mensaje diagnóstico acotado a 240 caracteres.

4.4 Confirmación y recuperación

Mensaje	Campos y restricciones
ACK_STATE	type="ACK_STATE", epoch, received, missing. received: hasta 32 rangos inclusivos [inicio,fin], ordenados, no solapados; 1 <= inicio <= fin <= 2147483647. missing: hasta 32 ids únicos del mismo dominio, ninguno dentro de received. Se permiten arreglos vacíos.
RELEASE, compatibilidad	type="RELEASE", deliveryId, status; status es displayed, discarded o failed. No contiene epoch.

received significa consumo terminal, no simple recepción del paquete binario. Una entrega descartada por obsolescencia también MUST confirmarse. missing solicita recuperar una entrega pendiente; no devuelve su crédito. El navegador actual utiliza ACK_STATE y emite un rango unitario por entrega terminal; no implementa agrupación automática de múltiples confirmaciones. RELEASE permanece soportado para herramientas anteriores. Ambos mecanismos son idempotentes: una entrega desconocida o ya confirmada no libera crédito otra vez. [C07]

5. Estructura binaria TILE

Cada mensaje WebSocket binario contiene exactamente una envoltura LUPA, sin concatenación de teselas:

Offset	Longitud	Contenido
0	4 bytes	H: uint32, big-endian
4	H bytes	Cabecera JSON, UTF-8 estricto
4 + H	payloadBytes	JPEG de una tesela

Longitud total E = 4 + H + payloadBytes
1 <= H <= 4096; 1 <= payloadBytes <= 262144
E máximo = 266244 bytes, sin cabecera WebSocket/TCP/IP

Campo de cabecera	Tipo / restricción
type	Literal TILE.
deliveryId, epoch	Enteros positivos hasta 2147483647.
imageId, imageVersion	Identidad de la versión abierta.
z	Índice válido en MANIFEST.
x, y	Índices enteros no negativos de columna/fila de tesela en z; no son píxeles del original.
w, h	Dimensiones reales del JPEG: 1..256, con bordes recortados según el manifiesto.
codec	Literal jpeg.
payloadBytes	Longitud exacta de los bytes que quedan tras la cabecera.
retry	Campo opcional; aparece en retransmisiones con valor 1 o 2. No aparece en el envío original.

El cliente MUST comprobar longitud, UTF-8, cabecera, versión, nivel, posición y tamaño del borde. El Worker verifica que la imagen decodificada tenga w/h declarados. Una cabecera incompatible no se dibuja. No hay checksum LUPA transmitido; TILE_SHA256 es diagnóstico local y no un campo del protocolo. [C09, C14, C16]

6. Selección y progreso de la vista

6.1 Elección de nivel

Sea W/H la imagen original, rw/rh el tamaño del rectángulo solicitado, vw/vh el viewport y Wz/Hz las dimensiones de un nivel. Se elige el menor z que cumpla simultáneamente:

rw * Wz >= vw * W
rh * Hz >= vh * H

Si ninguno alcanza esa densidad, se usa el nivel nativo. El candidato inicial es max(0, zAutomatico + detailOffset). El planificador prueba ese nivel y luego niveles menores hasta satisfacer ambos límites: no más de 512 descriptores y suma RGBA de teselas no-z0 <= bitmapBudgetBytes - 16 MiB. Cuenta 4*w*h por

tesela, incluidos bordes. z0 se cubre con la reserva. Si ningún candidato cabe, responde LIMIT_EXCEEDED. Esta degradación es explícita en PLAN, no una descarga de la imagen completa. [C08]

6.2 Geometría y prioridad

El rectángulo original se proyecta a z con piso en el inicio y techo en el extremo final. Dividir por 256 determina columnas y filas; los extremos semiabierto evitan incluir una tesela extra sobre un borde exacto. Dentro de cada grupo se recorre fila y luego columna. Se deduplican referencias (z,x,y) en el plan.

Para el eje x, la proyección y el rango de columnas son $lx0 = \text{floor}(x * Wz / W)$, $lx1 = \text{ceil}((x + rw) * Wz / W)$ y $\text{floor}(lx0 / 256) .. \text{floor}((lx1 - 1) / 256)$, después de limitar los extremos al nivel. El eje y usa el mismo cálculo con $H / Hz / rh$. El término -1 corresponde al extremo final exclusivo; no es un margen arbitrario. [C08]

En uniform se transmite la región visible al nivel aplicado. En focus, $\text{contextLevel} = \max(0, \text{appliedLevel} - 1)$. El radio en pantalla se transforma a dos radios en coordenadas originales: $rx = \text{radiusPx} * rw / vw$, $ry = \text{radiusPx} * rh / vh$. La prueba de intersección selecciona las teselas que intersectan la elipse, no solo aquellas cuyo centro está dentro.

Prioridad de un OPEN + VIEW en modo focus:

```
miniatura z0 -> contexto mínimo bajo el foco -> detalle del foco
               -> resto del contexto visible
```

La miniatura de apertura se agrega hasta que su primer envío quede comprometido. Un foco fuera de la región visible produce solo contexto y PLAN informa ese nivel efectivo. El servidor no deja el resto de la imagen vacío por atender el foco. El cliente pinta contexto antes de detalle. [C08, C17]

6.3 Consolidación de solicitudes

Durante pan/zoom, el navegador limita los VIEW transitorios a un intervalo nominal de 100 ms y usa detalle reducido ($\min(\text{offsetElegido}, -1)$). Después de 200 ms sin movimiento pide el detalle elegido. Deduplica firmas idénticas de rectángulo, viewport, modo, foco y detalle; redimensionar usa debounce de 180 ms. No hay un HTTP GET por tesela: los datos fluyen por el mismo WebSocket. Estos controles reducen intenciones y trabajo innecesario; no garantizan cero tráfico repetido. [C13]

6.4 Resumen del algoritmo de planificación

El pseudocódigo resume ViewPlanner.plan; no agrega controles al cable ni realiza I/O:

```
automatico = primer nivel con densidad suficiente en ambos ejes
              (si no existe, usar nivel nativo)
para candidato = max(0, automatico + detailOffset) hasta 0:
    construir region uniforme o contexto + foco
    agregar miniatura si corresponde; deduplicar (z,x,y)
    bytes = suma de 4*w*h de teselas no-z0
    si cantidad <= 512 y bytes <= bitmapBudgetBytes - 16 MiB:
        devolver el plan completo y sus niveles efectivos
fallar con LIMIT_EXCEEDED
```

Se reduce el nivel del plan completo, no se corta arbitrariamente una lista para enviarla incompleta. Eso conserva cobertura espacial dentro del presupuesto; PLAN permite al cliente conocer la definición que realmente se aplicó. [C08]

7. Crédito y cancelación

7.1 Contrapresión por consumo

Antes de encolar una entrega original se calcula E con la cabecera realmente codificada. Solo se reserva si $E \leq \text{freeWindowBytes}$, hay menos de 16 entregas pendientes y está disponible el presupuesto global de envolturas. Se guarda E en la reserva del delivery. Después de encolarla, escribirla o recibirla, el crédito todavía está ocupado. [C09]

Conexión activa:

$\text{freeWindowBytes} + \text{SUM}(E \text{ de reservas pendientes}) = \text{windowBytes}$

Ejemplo: W=1048576, H=220, JPEG=20000

E=20224; libre tras reservar=1028352

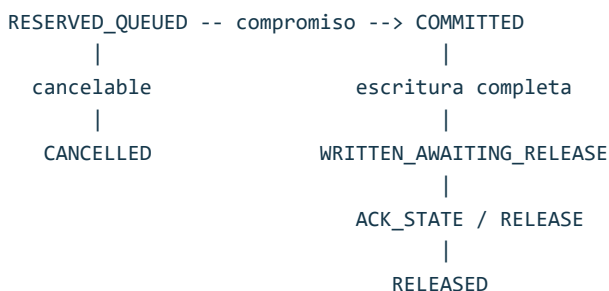
ACK terminal o cancelación antes de compromiso: devuelve 20224

ACK duplicado: devuelve 0

La escritura física y el crédito no tienen el mismo ciclo: terminar la escritura libera la envoltura transitoria, pero conserva la reserva lógica hasta confirmar consumo. Un ACK recibido se aplica a reservas conocidas de la época indicada; el servidor recorre su mapa acotado, no expande un rango de millones de ids. La invariante se verifica en transiciones; una inconsistencia provoca cierre 1011. No se aplica la ecuación a una sesión ya cerrada y limpiada. [C07, C09]

7.2 Sustitución de planes y carrera asíncrona

Aceptar otra intención invalida el plan previo, su permiso en espera y su tesela preparada. Cancela PLAN, DONE y entregas que aún estén únicamente en cola. Una entrega cancelada antes de compromiso devuelve exactamente su reserva. Una lectura antigua puede terminar, pero su generación se comprueba antes de usar el resultado normal y su lease se cierra. No se interrumpe a la fuerza una lectura de disco que ya corre. [C10]



Un mensaje comprometido MUST terminar su escritura para no romper el flujo WebSocket. Su crédito sigue reservado aunque cambie la época. El cliente descarta una tesela superada y confirma su época original. Cambiar de VIEW MUST NOT reiniciar la ventana ni reutilizar ids pendientes. Cancelación reduce tráfico obsoleto, pero no puede recuperar bytes ya enviados. [C10, C14]

7.3 Ciclo de envío y reanudación por eventos

En producción, cada plan normal encadena lectura, tesela preparada y escritura; no prepara indefinidamente nuevas teselas cuando la preparada espera crédito. `pump()` se reevalúa por callbacks y confirmaciones, sin esperar bloqueando un hilo. Este es su orden de decisión simplificado: [C25]

pump:

```

si cerrado, sin plan, plan ocupado o PLAN no comprometido: salir
si hay tesela preparada: intentarEnvio; salir
si el cursor termino: encolar DONE una sola vez; salir
si hay 16 entregas pendientes o un turno en espera: salir
solicitar turno DRR para la siguiente lectura normal

```

resultado de lectura:

```

devolver turno DRR al terminar almacenamiento
si generacion superada: cerrar lease JPEG; reevaluar pump
si fallo vigente: ERROR y terminar ese plan, sin DONE
conservar una tesela preparada; intentarEnvio

```

intentarEnvio:

```

calcular E con cabecera UTF-8 y JPEG reales
si E supera ventana total: ERROR
si E supera credito libre: conservar preparada y salir
reservar buffer global e id; registrar E; restarlo del libre
encolar TILE y cerrar el lease JPEG usado para construirlo

```

El límite de 16 también se comprueba al intentar enviar. El JPEG preparado ya puede ocupar memoria aunque aún no tenga reserva de crédito; la reserva de envoltura y el id definitivo se adquieren cuando puede enviarse. El permiso DRR cubre el trabajo de almacenamiento, no el tiempo de escritura ni la espera de ACK. [C09, C25]

Al completar la escritura se libera el buffer de envoltura, se arma el plazo de confirmación, se incrementa sentTiles del plan vigente y se reevalúa pump. ACK_STATE/RELEASE elimina la reserva lógica conocida, devuelve su E y reevalúa pump; hacerlo otra vez no devuelve nada. Una lectura antigua que termina libera su turno y permite intentar el plan nuevo. No hay mensaje de reanudación adicional. [C07, C09, C25]

El siguiente caso es ilustrativo, no una traza medida: una entrega de época 2 ya comprometida ocupa E=20224 en W=1048576. En el instante de sustituirla por VIEW de época 3, antes de reservar entregas nuevas, se conserva libre=1028352 y esa reserva. Si llega al cliente, este la descarta y confirma epoch=2; entonces libre vuelve a 1048576, suponiendo que no hubo otras reservas. Una confirmación duplicada no lo aumenta. Si hubiera estado solo en cola, la cancelación precompromiso habría devuelto E sin esperar al cliente. Son dos ramas distintas de la misma invariante.

8. STA: recuperación selectiva de entregas

Selective Tile Acknowledgment (STA) toma la idea de indicar bloques recibidos/faltantes de [RFC 2018](#), pero su unidad es una entrega JPEG de aplicación, no un segmento TCP. No hay temporizador de retransmisión automática por pérdida de red: TCP ya presta esa fiabilidad.

El servidor acepta missing=[d] solo si d sigue pendiente, su época coincide y ya fue escrito. Solicitudes duplicadas mientras se recupera se ignoran. Si la intención o la imagen ya no son vigentes al solicitarlo, libera la reserva obsoleta. En caso vigente relee solo la referencia de d, conserva el id y la reserva originales, y envía la misma identidad con retry=1 o retry=2. Cada reenvío reserva su envoltura transitoria, pero no crea otra reserva de ventana. El crédito representa entregas pendientes, no un límite de bytes acumulados retransmitidos. [C11]

La recuperación usa el executor de disco acotado, pero no pasa por la admisión DRR de lecturas normales. No debe afirmarse que DRR gobierna también todos los reintentos. Un reintento cuya época se vuelva antigua durante la lectura aún puede llegar; el receptor mantiene el descarte por identidad/época.

```
Servidor                               Cliente / Worker / Canvas
TILE d, epoch=e                        ->  intento de decodificar falla
                                     <-  ACK_STATE e: missing=[d]
TILE d, epoch=e, retry=1 ->          acepta id repetido esperado
                                     decode y drawImage
                                     <-  ACK_STATE e: received=[[d,d]]
devuelve crédito E de d
```

El límite es dos retransmisiones por entrega. El cliente solo acepta un id repetido si esperaba recuperación. Si agota su política, termina como fallo y confirma consumo; si el servidor recibe una petición más allá del límite, libera la reserva y emite INTERNAL_READ_ERROR. Errores de lectura/cola se notifican, y el plazo de confirmación vuelve a armarse cuando corresponda. DONE no espera estos intentos ni cuenta los reenvíos. [C07, C11, C15]

Una traza real conservada muestra 22 TILE recibidas, 21 dibujos, un intento fallido y 21 confirmaciones únicas, para un DONE de 21 originales. Por ello RELEASE == TILE recibidas no es una invariante general con recuperación. Véase [evidencia STA del 23 de septiembre](#).

9. Recursos, caché y equidad

9.1 Controles del servidor

El estado de sesión se modifica serialmente. Los callbacks de red no realizan la lectura de teselas ni la apertura de metadatos en disco: usan executors separados con colas limitadas. La cola de escritura mantiene como máximo una operación socket.write pendiente y reanuda escrituras parciales. [C04, C18]

Recurso	Valor base / unidad
Conexiones / WebSockets / sesiones LUPA	128 / 64 / 32
Executor general de estado/HTTP	min(8,CPU) hilos + cola compartida de 256 tareas
Cola serial por sesión	64 tareas en espera; una tarea activa a la vez
Grupo de callbacks de red	min(4,CPU) hilos; no sustituye los executors de almacenamiento
Disco / metadatos	min(4,CPU) hilos + cola 64 / min(2,CPU) hilos + cola 32
Admisión de lecturas normales	8 turnos admitidos entre ejecución y cola de disco; hasta 32 owners esperando turno
Quantum DRR	131072 bytes (128 KiB)
Caché JPEG residente compartida	134217728 bytes (128 MiB)
Envolturas transitorias compartidas	16777216 bytes (16 MiB)
Cola de escritura WebSocket	64 mensajes/tramas en cola por conexión
Entregas pendientes por conexión	<= 16 y <= ventana negociada en bytes

Los parámetros configurables se fijan al arrancar; las constantes internas se identifican aparte. -Xmx controla heap JVM, no el RSS completo ni los presupuestos de aplicación. La imagen original no se carga completa para cada VIEW: se leen teselas publicadas. No se deduce de esta tabla una cota exacta de toda la memoria del proceso. [C19]

Ocho turnos admitidos no significan ocho hilos leyendo físicamente en paralelo: el executor de disco tiene hasta cuatro hilos por defecto. El límite del turno normal es independiente de la cola de disco y no incluye la relectura selectiva. La cola serial de 64 es una constante interna, no un argumento de arranque; los demás parámetros de ejecución siguen su configuración correspondiente. [C23, C24, C26]

La configuración exige sesiones $\leq$ WebSockets $\leq$ conexiones, caché capaz de alojar al menos un payload máximo y buffers transitorios $\geq$ $\text{maxSessions} * 266244$. También exige $\text{maxTileReads} \leq \text{diskThreads} + \text{diskQueueCapacity}$ y quantum de 1..262144. Estas comprobaciones evitan configuraciones incoherentes; no garantizan ausencia de saturación durante cualquier carga. [C19]

Saturación	Política implementada
Capacidad WebSocket / trabajo HTTP	HTTP 503 antes del upgrade o al rechazar trabajo HTTP.
Sesiones LUPA	HELLO: ERROR LIMIT_EXCEEDED y cierre 1013.
Cola de metadatos	OPEN: IMAGE_NOT_READY, sin aceptar la nueva época.
Turnos normales / cola de disco	Plan vigente: INTERNAL_READ_ERROR, sin DONE ficticio. Un owner duplicado por lectura anterior se difiere, no se trata como cola llena.
Presupuesto transitorio de TILE	LIMIT_EXCEEDED; no se acumulan envolturas ignorando el límite.
Cola serial / escritura WebSocket	Rechazo de trabajo de sesión o envío: cierre 1011 y limpieza.

Los executors de almacenamiento rechazan con AbortPolicy; no ejecutan la lectura en el callback solicitante mediante CallerRunsPolicy. Esta decisión mantiene separado el progreso de red del trabajo potencialmente bloqueante. [C23, C24]

9.2 Variante DRR implementada

Cada sesión tiene como máximo un turno normal activo o en espera. Con competencia, el scheduler visita owners en orden circular, suma Q al déficit hasta 4Q y concede el permiso si cubre el costo estimado. Si no alcanza, rota. El costo inicial es Q; al terminar una lectura usa $\min(\text{JPEGreal}, 4Q)$ como estimación siguiente y lo resta del déficit, con piso -4Q. La métrica chargedBytes acumula el JPEG real completo. Sin competencia puede conceder inmediatamente un permiso libre. [C12]

Esto es equidad aproximada por costo comprimido observado, no DRR de paquetes con tamaño conocido antes de servirlos, ni garantía de latencia idéntica. No gobierna las prioridades dentro del plan. Un cliente sin crédito no monopoliza turnos útiles; otro que no lee queda aislado por la cola de escritura y su timeout. La prueba determinista observa A(220 KiB) -> B(32 KiB) -> C(128 KiB) -> B(32 KiB) -> A(220 KiB). La [evidencia DRR real](#) registra ambas sesiones progresando y una lectura global con Q=8192. Es diagnóstico funcional, no benchmark de throughput.

9.3 Políticas de caché

Lugar	Política y alcance
Servidor, JPEG	LRU global por (imageId,imageVersion,z,x,y). Leases comparten bytes inmutables; al expulsar una entrada aún en uso sus bytes quedan contabilizados aparte hasta cerrar el último lease. Puede haber lecturas duplicadas ante misses concurrentes; no hay single-flight.
Cliente, ImageBitmap	Clave con imagen/versión/z/x/y. Al cambiar época cierra teselas no-z0; retiene z0 como base. OPEN/reset cierra todas. No es una LRU persistente entre vistas.
HTTP catálogo	no-store; refrescar obtiene las versiones publicadas actuales.

La LRU ahorra lecturas, no evita por sí misma retransmitir una tesela al navegador. El protocolo no contiene HAVE_TILE, ETag de teselas ni anuncio de inventario cliente. No se puede atribuir a la caché HTTP del navegador la reutilización del binario WebSocket. [C01, C17, C20]

9.4 Receptor acotado y liberación

El navegador reserva $4 \cdot w \cdot h$ antes de decodificar. Su perfil es 64 MiB total: hasta 16 MiB para contexto más reservas transitorias y hasta 48 MiB para detalle almacenado. Transfiere el ArrayBuffer al Worker; el Worker usa como máximo dos decodificaciones activas, 32 trabajos en cola y 8 MiB de JPEG en cola. La geometría efectiva del bitmap se comprueba tras createImageBitmap. [C15, C16]

Las guardas se aplican antes y después del trabajo asíncrono: conexión, época e identidad de versión. Un ImageBitmap obsoleto se cierra con close(); no se incorpora al compositor. Tras pintarlo se confirma la entrega, pero el bitmap puede continuar almacenado hasta el cambio de vista. Liberar crédito no equivale a cerrar inmediatamente el bitmap. Los contadores describen memoria administrada por LUPA; no incluyen toda la RAM/GPU del navegador ni copias internas del codec. [C14-C17]

10. Fallos, plazos y reconexión

ERROR LUPA	Condición representativa / efecto
VERSION_UNSUPPORTED	Versión HELLO positiva, dentro del rango del campo, distinta de 1; no se negocia la sesión.
BAD_VIEW	Geometría, identidad o parámetros de VIEW inválidos; no sustituye el plan anterior.
IMAGE_NOT_FOUND	Identificador inválido o ausente en el catálogo.
IMAGE_NOT_READY	Metadatos de apertura no disponibles/válidos o admisión de su trabajo saturada.
LIMIT_EXCEEDED	Ventana/presupuesto/admisión/límites de selección insuficientes.
INTERNAL_READ_ERROR	Fallo al leer/encapsular/enviar una tesela o recuperación agotada.

Un ERROR de un plan no genera un DONE ficticio. El visor ignora respuestas superadas y presenta un error recuperable cuando procede. JSON inválido, mensaje desconocido o secuencia prohibida son violaciones de política y cierran con 1008. [C04, C06, C11]

Temporizador base	Inicio del plazo y consecuencia
Cabeceras HTTP: 5 s	Desde la recepción inicial de conexión; petición incompleta expira.
HELLO: 5 s	Al abrir WebSocket; sin negociación válida, cierre 1008.
Ping / Pong: 15 s / 10 s	Se envía Ping; plazo Pong empieza tras escribirlo completo y exige el mismo nonce. Timeout: 1001.
Confirmación terminal: 30 s	Tras escribir completamente la entrega; sin ACK_STATE/RELEASE, cierre 1008 y limpieza. Se rearma tras reenvío completo.
Progreso de escritura: 10 s	Sobre escritura comprometida; avanza al escribir bytes positivos; falta de progreso provoca 1011.
Cierre WebSocket: 2 s	Espera limitada del cierre iniciado antes de cerrar transporte.

Tramas inválidas producen 1002, UTF-8 inválido 1007 y exceso de tamaño 1009. Rechazar HELLO por saturación de sesiones notifica LIMIT_EXCEEDED y cierra con 1013. 1006 es el estado local observado ante cierre anormal; MUST NOT enviarse como código Close en el cable. Ping/Pong son tramas WebSocket, no controles JSON LUPA. [C03, C18, C19; RFC 6455 sección 7.4]

Al cerrar se cancelan planes y timers, se devuelven permisos y leases, y se limpia la contabilidad pendiente. Reconectar abre otro WebSocket y repite HELLO/WELCOME/OPEN; no reanuda ids, ventana ni épocas de la conexión vieja. El cliente conserva la selección, pero invalida sus callbacks viejos y limpia Canvas/budget/ledger. Esta recuperación explícita evita presentar una imagen congelada como si la sesión siguiera viva. [C21]

11. Validación, resultados y límites de las afirmaciones

11.1 Evidencia disponible

La tabla vincula las garantías del protocolo con su implementación y pruebas concretas. Los enlaces de prueba llevan a su método/aserciones en el commit fijado; no constituyen por sí solos el log de una ejecución. Los resultados de campaña se conservan en A23/I23/E23/E24 y se distinguen abajo. No se presenta esta revisión documental como una nueva ejecución del producto.

Garantía / alcance	Implementación	Prueba concreta: qué comprueba
Validar controles sin aceptar una vista inválida	LupaJson, handleView [C05, C06]	JSON duplicado y BAD_VIEW sin consumir época.
Crédito exacto e idempotencia	trySendPending, acknowledgeDelivery [C07, C09]	Pausa, reanudación y confirmación duplicada; secuencia de invariante.
Cancelar cola, pero conservar entrega comprometida	invalidatePlan, write handles [C10, C18]	Precompromiso: devuelve crédito y no reutiliza id; poscompromiso: espera confirmación, tolera callback tardío.
Resultado antiguo no emite TILE del plan sustituido	generación y onTileRead [C10, C25]	Lectura de época superada. La guarda de Canvas se comprueba además en A23.

Garantía / alcance	Implementación	Prueba concreta: qué comprueba
STA conserva id y reserva	requestSelectiveRetransmit [C11]	Recuperación con mismo id y sin crédito extra, más traza real de esta sección.
Cobertura dentro del presupuesto y prioridad de foco	ViewPlanner [C08]	Reducir el plan completo y contexto/foco sin duplicados.
Costo JPEG influye en turnos normales	TileReadAdmission [C12]	DRR A-B-C-B-A, más traza DRR de dos navegadores.
Separar cliente sin ACK de cliente que no lee	crédito, admisión y escritura [C09, C18]	Sin RELEASE, el otro termina; socket real que no lee.
Permiso de lectura y buffer tienen ciclos distintos	read lease, TransientBufferBudget [C24, C25]	Permiso devuelto tras almacenamiento; buffer hasta escritura.
Expirar entregas no confirmadas y limpiar al reconectar	timers y cleanup [C18, C21]	Entrega escrita más antigua expira; cierre idempotente y conexión nueva.
Presupuesto cliente y ERROR no confundido con DONE	BitmapBudget, LupaClient [C14, C15]	Reserva/liberación RGBA; plan fallido sin DONE.

La aceptación real [A23](#) reporta PASS de 100 interacciones y 100 ciclos -2 -> 0 -> -2; cero dibujos obsoletos, mezclas de identidad y solicitudes externas; pico RGBA observado de 3342336 B. [I23](#) reporta integración, cliente lento/no lector y reconexión PASS; ambos identifican el artefacto `a23-i23-evidence-f707ac02136be4922dead8d707e014f37863e98b`. La regresión se documenta en [cierre E23](#). [E24](#) conserva 120 ejecuciones medidas, 1/5/20 clientes, cinco repeticiones y `suiteFailures=0` sobre `c9c7f0a19095d635486d52c0e2f8ab78d3e3e25b`. Son alcances y revisiones identificados, no un PASS nuevo generado por esta redacción.

Extracto de la [traza real STA, 23 de septiembre](#). Se muestran hora UTC y campos relevantes, omitiendo otros eventos; no se altera su orden:

```
16:59:00.035 IN    TILE           delivery=1 epoch=2
16:59:00.036 OUT  ACK_STATE      received=[] missing=[1]
16:59:00.036 LOCAL FAILED         delivery=1 (fallo sintetico)
16:59:00.040 IN    TILE           delivery=1 epoch=2 retry=1
16:59:00.040 LOCAL RETRANSMIT_ACCEPTED delivery=1 epoch=2
16:59:00.056 OUT  ACK_STATE      received=[[1,1]] missing=[]
16:59:00.082 IN    DONE           epoch=2 sentTiles=21
16:59:00.082 LOCAL DONE_STATE    decode=5 pintura=0
```

La falla fue inyectada mediante el hook diagnóstico; no demuestra pérdida de paquetes TCP. DONE llegó con cinco decodificaciones pendientes, corroborando que no certifica imagen ya visible. La traza termina con 22 intentos recibidos, 21 dibujos, un intento fallido y 21 confirmaciones únicas: el contador histórico `releases` registra `ACK_STATE`; el contador `terminal` suma resultados de intentos, no ids distintos. La igualdad de crédito durante el reintento la comprueba la prueba de sesión enlazada, no este log del navegador.

Para cancelación se usa evidencia determinista, no se inventa una captura de red. Estas aserciones pertenecen a las pruebas pre/poscompromiso enlazadas en la tabla:

```
// precompromiso, extracto de LupaSessionTest:
assertTrue(sender.firstSend.cancelled);
assertEquals(3, firstVisible.get("epoch").asInt());
assertEquals(2, firstVisible.get("deliveryId").asInt(),
    "cancelled deliveryId=1 is never reused");
// poscompromiso, despues de VIEW nuevo:
assertEquals(1, session.snapshotForTest().pendingDeliveries());
assertCreditInvariant(session);
```

La primera prueba verifica que la entrega cancelada id=1 no aparece ni se recicla; la segunda conserva la reserva anterior hasta RELEASE y vuelve a comprobarla ante callback tardío y confirmación duplicada. E24 complementa estas pruebas con el efecto medido de cancelar trabajo superado, sin convertir la prueba controlada en una medición de latencia real.

11.2 Efecto medido de cancelación

Extracto de E24, variante normal, 20 clientes, cinco repeticiones por condición:

Carga / caché	Primera TILE mediana / p95 ms	DONE mediana / p95 ms	Ahorro JPEG frente a no-cancel
Stable / cold	97.512 / 98.983	196.641 / 203.260	Sin ahorro: mismas medianas de bytes
Stable / warm	11.904 / 15.453	79.639 / 88.865	Sin ahorro: mismas medianas de bytes
Aggressive / cold	43.939 / 51.827	109.480 / 133.061	92.51%
Aggressive / warm	29.601 / 40.148	62.087 / 68.390	84.76%

cold significa JVM nueva y LRU LUPA vacía, no caché de páginas del sistema operativo vacía. warm conserva proceso y calentamiento no medido. Las latencias agrupan observaciones elegibles y p95 usa nearest-rank; el ahorro es la mediana de comparaciones emparejadas. Bytes JPEG y bytes de aplicación son métricas distintas. [C22]

La baseline sin cancelación es una propiedad experimental local, desactivada en producción; no agrega mensajes al protocolo. En aggressive/warm con 20 clientes aún hubo 51.08% de bytes obsoletos en normal: bytes enviados antes de que la nueva intención pudiese cancelarlos. Eso no contradice la regla de no dibujarlos. La cancelación evidencia un ahorro de trabajo superado, no ausencia absoluta de tráfico obsoleto.

11.3 Metas visuales y límites conocidos

Las metas de primera vista < 1 s y refinamiento < 500 ms se midieron en Canvas en A24. La sección 16 detalla diez corridas, entorno, definición y resultados; no se sustituyen con VIEW->TILE ni VIEW->DONE. Las metas se cumplieron en el alcance medido, sin afirmar latencia física de pantalla, todos los equipos ni todas las cargas.

La escala funcional reportada incluye eval-93, 176393 x 176393 y 635214 teselas validadas; no certifica rendimiento para cualquier archivo denominado 40/80 GB. El perfil no tiene autenticación, TLS nativo, adaptación al ancho de banda, actualización dinámica de ventana ni reanudación de sesión. La recuperación selectiva no está arbitrada por DRR. El mapa de historial de costos DRR y la guarda por conexión del Worker no tienen purga explícita en sus clases; la campaña finita no prueba consumo constante tras reconexiones indefinidas. Estos son límites de alcance, no funciones que se deban atribuir al protocolo. [C12, C16]

12. Trazabilidad al código y referencias

Los enlaces siguientes fijan el commit de implementación. La búsqueda por nombre de método sigue siendo útil si posteriormente se mueve una línea. Este documento es la entrada consolidada; contrato-v1.md y documentos de etapas anteriores se conservan como historial y material complementario.

Id	Punto de implementación
C01	HttpRouter.route/catalog
C02	WebSocketHandshake.evaluate
C03	WebSocketFrameParser y WebSocketTextAssembler.accept
C04	LupaSession.handleHello y despacho
C05	LupaSession.handleOpen/onOpenResolved y ViewRequest.parse
C06	LupaJson.parseControl/requireOnly y LupaProtocol
C07	LupaSession.handleAckState/acknowledgeDelivery , handleRelease y cliente release
C08	ViewPlanner.plan/automaticLevel/buildPlan
C09	LupaSession.trySendPending/envelope e invariante de crédito
C10	LupaSession.invalidatePlan/cancelUncommittedDeliveries
C11	LupaSession.requestSelectiveRetransmit
C12	TileReadAdmission.acquireOrQueue/release/selectNextWaiterLocked
C13	LupaClient.queueNavigationView/requestView
C14	tileEnvelope.parseTileEnvelopeBase/validateTileAgainstManifest y LupaClient.onBinary/onWorker
C15	BitmapBudget.reserve/commit y DeliveryLedger.releaseOnce
C16	decodeWorker límites y decode
C17	CanvasCompositor.beginEpoch/store/paint/reset
C18	WebSocketWriteQueue.requestDrive , WebSocketHeartbeat y LupaSession timers
C19	ServerConfig.defaults
C20	CompressedTileCache.acquire/evictUntilFits/release
C21	LupaClient.connect/cleanupConnection y LupaSession.onClosed
C22	summarize-campaign.py y analyze-matrix.py
C23	SerialExecutor : capacidad y orden por sesión y submitSerial : rechazo
C24	StorageExecutors : colas y AbortPolicy y NioHttpServer : admisión/executor general
C25	LupaSession.pump/requestTileRead y onTileRead : guarda y tesela preparada
C26	LupaApplication : recursos compartidos e inyección de executors

Referencias técnicas: RFC 6455, The WebSocket Protocol; RFC 9293, TCP; RFC 2018, TCP Selective Acknowledgment Options; RFC 2119 y RFC 8174, terminología normativa. Se usan como referencias acotadas, no como declaraciones de implementar TCP o registrar un estándar.

Apéndice A. Intercambio concreto de ejemplo

Ejemplo ilustrativo, no captura de ejecución: imagen demo/v1 de 4096 x 3072, cinco niveles. Las épocas de OPEN y VIEW son distintas. El primer JPEG tiene 9000 bytes; H se calcula sobre los bytes UTF-8 reales de su cabecera, no sobre el número de caracteres.

```
{ "type": "HELLO", "version": 1, "windowBytes": 1048576, "bitmapBudgetBytes": 67108864 }

{ "type": "WELCOME", "version": 1, "windowBytes": 1048576, "maxTileBytes": 262144, "maxInFlight": 16 }

{ "type": "OPEN", "epoch": 1, "imageId": "demo" }

{ "type": "MANIFEST", "epoch": 1, "imageId": "demo", "imageVersion": "v1",
  "width": 4096, "height": 3072, "tileSize": 256, "levels": [
    { "z": 0, "width": 256, "height": 192 },
    { "z": 1, "width": 512, "height": 384 },
    { "z": 2, "width": 1024, "height": 768 },
    { "z": 3, "width": 2048, "height": 1536 },
    { "z": 4, "width": 4096, "height": 3072 } ] }

{ "type": "VIEW", "epoch": 2, "imageId": "demo", "imageVersion": "v1",
  "rect": { "x": 0, "y": 0, "width": 4096, "height": 3072 },
  "viewportPx": { "width": 1024, "height": 768 },
  "detailOffset": 0, "mode": "uniform", "focus": null }

{ "type": "PLAN", "epoch": 2, "appliedLevel": 2, "contextLevel": 2 }
```

Cabecera dentro del primer TILE binario, seguida de 9000 bytes JPEG:

```
{ "type": "TILE", "deliveryId": 1, "epoch": 2, "imageId": "demo",
  "imageVersion": "v1", "z": 0, "x": 0, "y": 0, "w": 256, "h": 192,
  "codec": "jpeg", "payloadBytes": 9000 }
```

Confirmación tras consumo terminal:

```
{ "type": "ACK_STATE", "epoch": 2, "received": [ [ 1, 1 ] ], "missing": [] }
```

La región z2 ocupa 4 x 3 teselas; con la miniatura son 13 originales. Después de escribirlos:

```
{ "type": "DONE", "epoch": 2, "sentTiles": 13 }
```

DONE puede llegar con decodificaciones/pinturas pendientes. Para focus, un VIEW posterior puede conservar rect/viewport, usar epoch=3, mode="focus" y focus={ "x": 2048, "y": 1536, "radiusPx": 144 }. Para comparar protocolos, MUST conservarse por separado la cantidad de originales, intentos retransmitidos, consumo terminal y vistas canceladas.

13. Versión de implementación y validación

6 de octubre de 2026. Erwin Arevalo, 23001717; Adrian Romero, 23004161. La especificación de Adrian se conserva en las secciones 1-12. Esta revisión integra arquitectura, almacenamiento, evidencia visual y mediciones, sin cambiar mensajes ni algoritmos del protocolo.

13.1 Identidad del candidato y corrección del visor

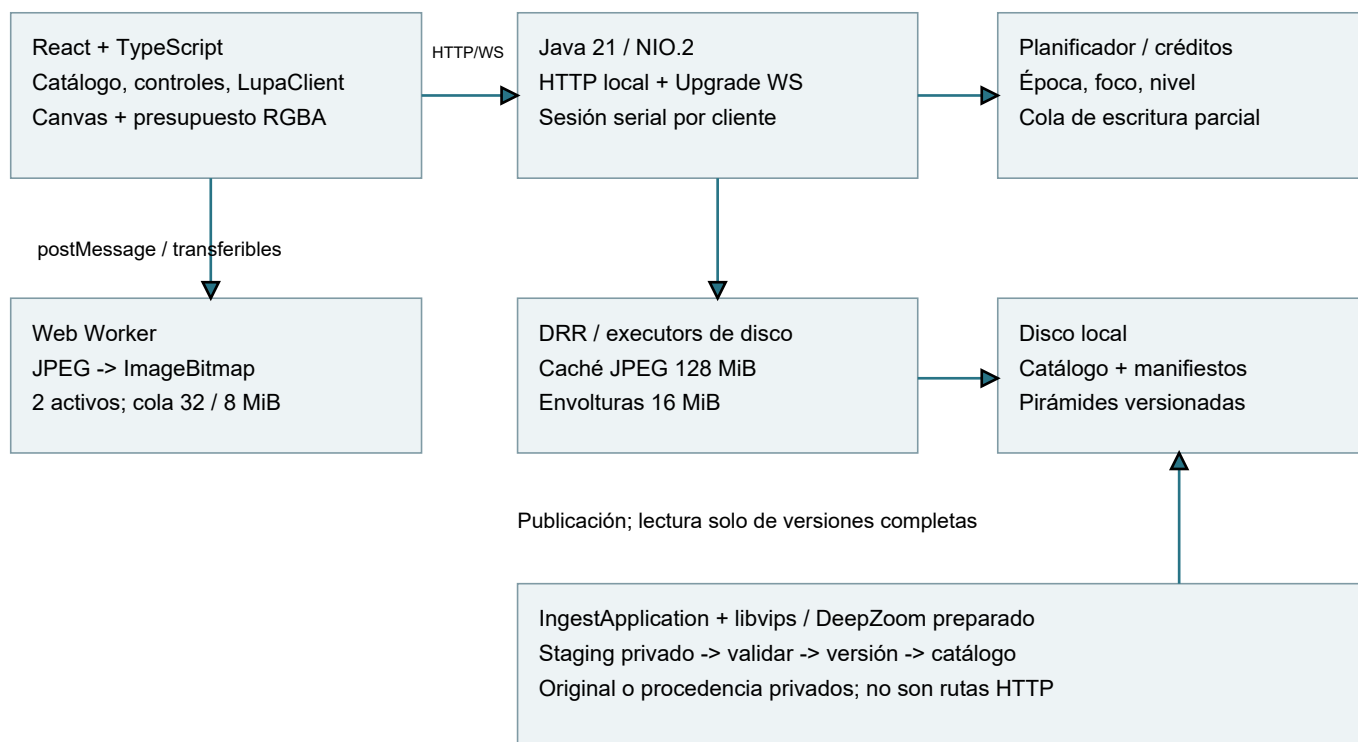
Elemento	Identidad / resultado
Base local y remota	f57fb91465ad0f9d1e9d783d0deb5e9453899bd8; origin/main coincidía al iniciar.
Corrección A24	El índice HTML servía app-Dyp4zZvL.js de l22. Ahora referencia app-E9QJGLo1.js, igual al generado por el código fuente actual. No se cambió la lógica de frontend/backend.
Huella del JS actual	SHA-256: 0d3952d3ae501905ce931f114818f08530008a54b38044cbc0e396bce0fbfdaf
Identificación reproducible	docs/evidence/a24/candidate-files.sha256 conserva índice, assets, JAR y scripts. Es base + ajuste local; no se inventa un nuevo commit.
Regresión y compilación	Maven offline: 179 pruebas, 0 fallos/errores. Frontend: 48 pruebas, 9 archivos; compilación TypeScript/Vite correcta.
Aceptación en navegador	100 cambios de vista y 100 ciclos de detalle PASS; carrera de decode tardío PASS sobre el visor actualizado.

13.2 Qué evidencia es nueva

A24 agrega diez recorridos de Canvas (cinco por muestra), capturas, trazas por entrega, reducción de memoria, importación válida/inválida y comprobaciones HTTP. E24 mantiene sus 120 ejecuciones backend históricas: no se presentan como ejecutadas otra vez.

La medición preliminar que contaba repintados posteriores quedó descartada. El runner final contabiliza solo el primer dibujo de cada entrega y limita la observación a su fase. Las corridas previas con el asset antiguo tampoco son la evidencia de cierre. El conjunto válido es docs/evidence/a24 y results/a24/candidate-final.

14. Arquitectura implementada



El diagrama agrupa responsabilidades; las flechas señalan comunicación o dependencia. La publicación alimenta el disco, no la sesión de un navegador. El cliente nunca recibe una ruta privada al original.

14.1 Recorrido y separación de trabajo

1. Java entrega página, recursos y catálogo. El navegador selecciona una imagen y abre WebSocket.
2. OPEN fija una versión; VIEW describe región, viewport, detalle y foco. ViewPlanner produce el plan en memoria.
3. La sesión solicita admisión de lectura normal. El trabajo de disco/metadatos corre en ejecutores acotados, no en callbacks de red.
4. Se lee JPEG publicado o caché, se reserva crédito y envoltura y se encola TILE. Solo una escritura por socket progresa a la vez.
5. El navegador valida identidad y formato, transfiere el payload al Worker, reserva RGBA y compone en Canvas.
6. La confirmación terminal devuelve crédito. El bitmap puede seguir almacenado hasta cambiar vista; al invalidarlo se cierra.

14.2 Asincronía y decisiones

Serializar el estado de cada sesión evita carreras sin mezclar las vistas de clientes. Compartir JPEG comprimido evita duplicar innecesariamente caché. Separar leases JPEG, buffers transitorios y créditos evita confundir memoria con consumo remoto. La lectura selectiva STA usa el executor acotado, pero no el turno DRR normal. [C23-C26]

15. Procesamiento y almacenamiento

15.1 Importador ordinario: JPEG / TIFF

El administrador ejecuta IngestApplication localmente. Preflight comprueba herramientas, archivo, dimensiones y espacio estimado. Bajo bloqueo de importación se asigna la siguiente versión y se prepara un directorio de trabajo privado. libvips aplica orientación, conversión sRGB y aplanado; genera DeepZoom JPEG de 256x256, sin solapamiento y con profundidad onetile. Existen archivos intermedios en staging: no se afirma que la ingestión no ocupe disco.

PyramidMath divide cada dimensión entre dos con techo hasta que ambas caben en 256; invierte la lista para que z0 sea el nivel mínimo. Por nivel: columnas=ceil(ancho/256), filas=ceil(alto/256). Los bordes conservan su tamaño real. Java valida todas las teselas: archivo regular sin symlink, tamaño 1..262144, JPEG decodificable, dimensiones previstas y RGB.

15.2 Publicación y originales

Ruta / política	Contenido y regla
originals/id/version	Original privado en modo copy, o metadatos/referencia privada en modo reference. La ruta ordinaria calcula SHA-256; reference evita duplicar el archivo y exige conservar su ubicación.
staging/job	Intermedios y versión aún no publicada. Un fallo no incorpora una pirámide parcial al catálogo.
pyramids/id/version	manifest.json y tiles/z/x_y.jpg; versión completa que no se sobrescribe mediante el importador.
catalog.json	Se reemplaza atómicamente después de publicar la versión. Un fallo posterior puede dejar una versión completa huérfana; el catálogo anterior sigue válido. Refrescar /api/catalog descubre nuevas imágenes.

15.3 Ruta especial de la muestra oficial PNG gigante

scripts/eval/stream_png_to_dz.py consume bytes PNG secuencialmente mediante pyvips y produce la pirámide DeepZoom en disco. No materializa el PNG completo, pero sí almacena las teselas. IngestApplication import-prepared-dz enlaza o copia exclusivamente las teselas esperadas a staging, crea un manifiesto y las decodifica/valida antes de publicar. Los hardlinks comparten contenido: el administrador debe mantener inmutable la fuente preparada.

eval-93/v1: 176393x176393, 635214 teselas y 28236373947 bytes JPEG según metadatos conservados. El PNG declarado tiene 93493762699 bytes. Su SHA del archivo comprimido procede de metadatos de origen; la validación local de teselas no equivale a recalcular ese SHA sobre un archivo original ausente. Esta precisión evita atribuir a la ruta preparada las garantías de copia de la ruta ordinaria.

16. Campaña visual A24: método y resultados

Chrome 154.0.8037.98 headless en Windows 11; servidor Java 21.0.12.1 en Ubuntu/WSL2, loopback. AMD Ryzen 9 8940HX, 32 procesadores lógicos, 31.05 GiB de RAM física. Viewport 1440x1000, DPR 1. Un cliente, cinco repeticiones por imagen. HTTP cache deshabilitada; LRU Java conservada entre corridas y caché del SO no vaciada. No es una comparación cold/warm.

Inicio: clic de selección o clic de detalle -2 a 0. **Fin:** primer drawImage de todas las entregas del plan vigente; se exige que los deliveryId únicos coincidan con sentTiles. La métrica adicional usa doble requestAnimationFrame después de esos dibujos como oportunidad de presentación. No acredita percepción humana ni el instante físico de refresco de la pantalla.

Se excluyen repintados repetidos de la miniatura, pausas del runner y generaciones anteriores. p95 usa nearest-rank: con n=5 coincide con el máximo; es una muestra pequeña, no una estimación universal de producción.

Muestra / métrica	drawImage mediana / p95 (ms)	Doble rAF mediana / p95 (ms)
demo-grande / vista inicial	35.0 / 523.6	52.9 / 549.7
demo-grande / refinamiento	21.2 / 34.3	34.9 / 47.9
eval-93 / vista inicial	26.6 / 40.7	37.6 / 84.7
eval-93 / refinamiento	11.3 / 12.6	25.1 / 26.1

Resultado: las diez corridas cumplen <1 s para completar el dibujo inicial y <500 ms para refinar, también bajo el proxy doble rAF. Esto cierra la medición pendiente de §11.3 únicamente para este entorno, viewport y caché. eval-93 no es necesariamente más lenta por ser mayor: la vista completa selecciona pocos niveles/teselas, no todos los píxeles del original.

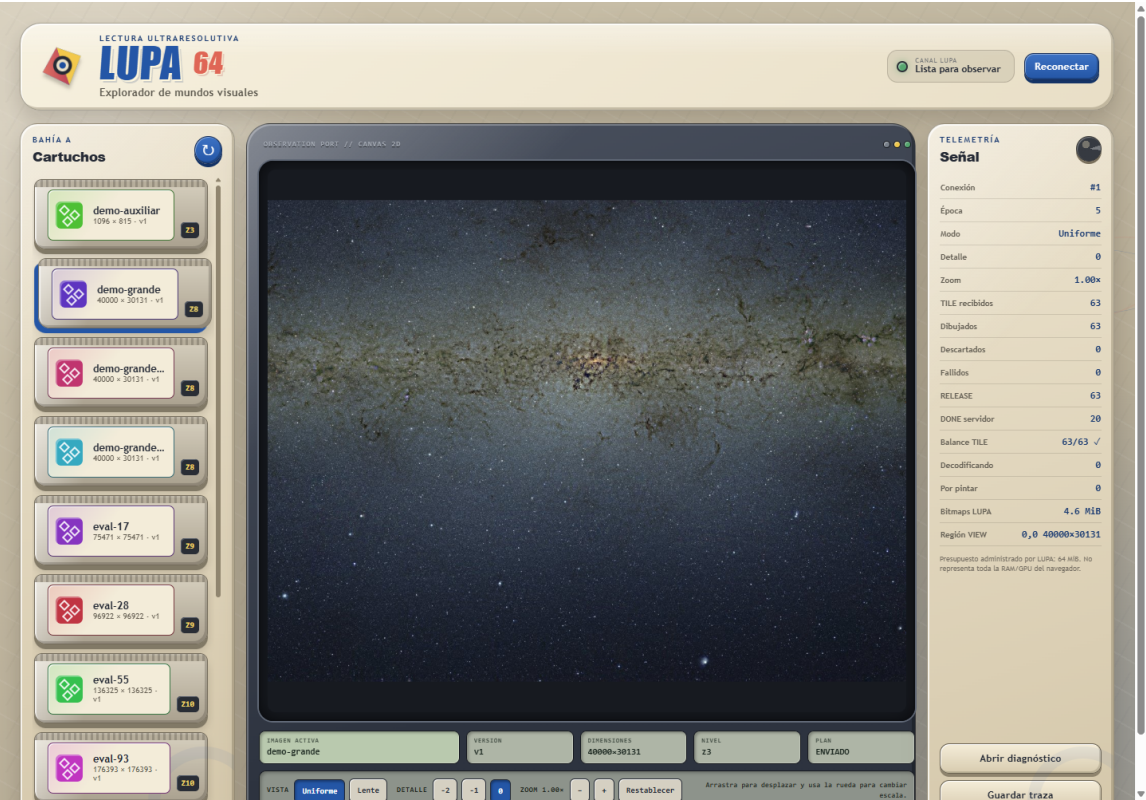
16.1 Corrección y recursos

Comprobación	Resultado real
10 recorridos de selección/detalle/foco	10 PASS; 0 dibujos obsoletos, 0 mezcla, 0 fallos decode y 0 solicitudes externas.
demo-grande: reducir z3 -> z1	4784104 -> 369576 bytes RGBA vivos; 20 bitmaps cerrados por reducción.
eval-93: reducir z2 -> z0	2024116 -> 119716 bytes RGBA vivos; 10 bitmaps cerrados por reducción.
Aceptación completa repetida	100 interacciones + 100 ciclos PASS; pico 4784104 B, 21 bitmaps vivos, 16 trabajos pendientes; carrera tardía PASS.

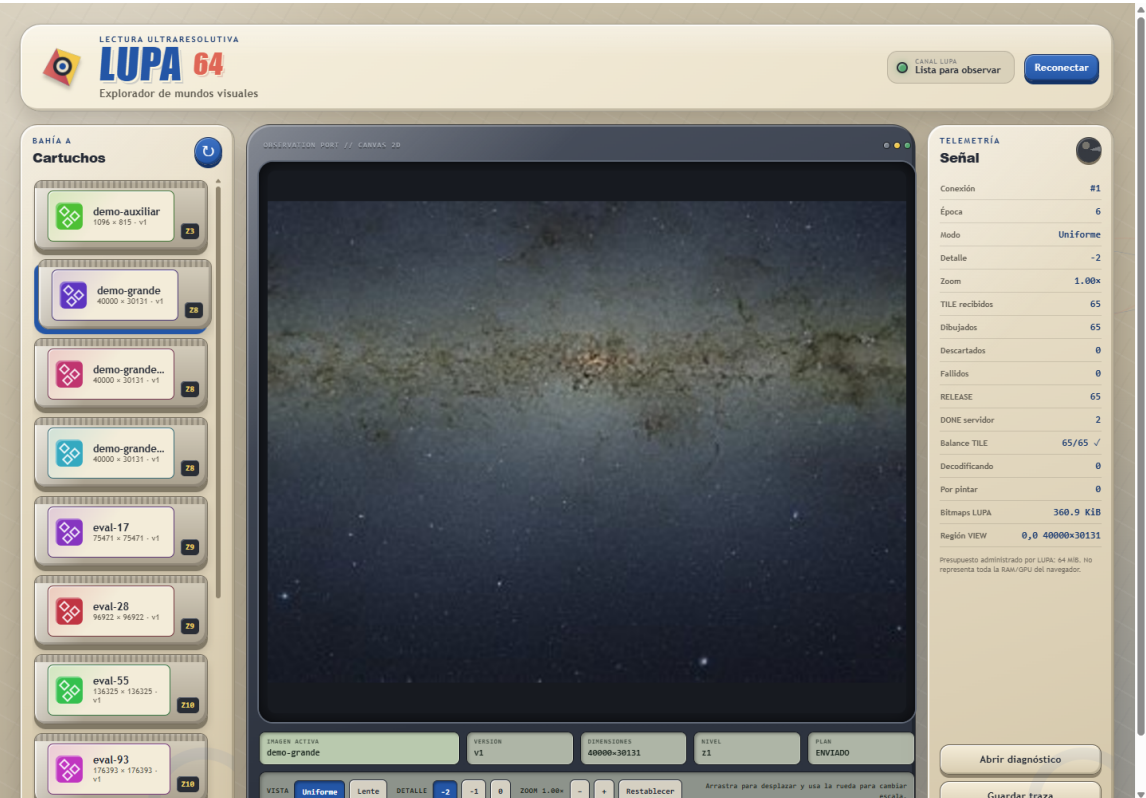
Los contadores son de recursos administrados por LUPA, no RSS/GPU. El runner visual bloqueó salida externa del navegador mediante un proxy local inalcanzable y permitió loopback. La red del equipo no se desactivó; no se llama a esta prueba una desconexión física de Internet.

17. Evidencia del visor y liberación

Capturas reales de demo-grande/v1 (40000x30131), repetición 1 del candidato actualizado. La tabla de §16 respalda los números; las capturas muestran el estado visible y los contadores.

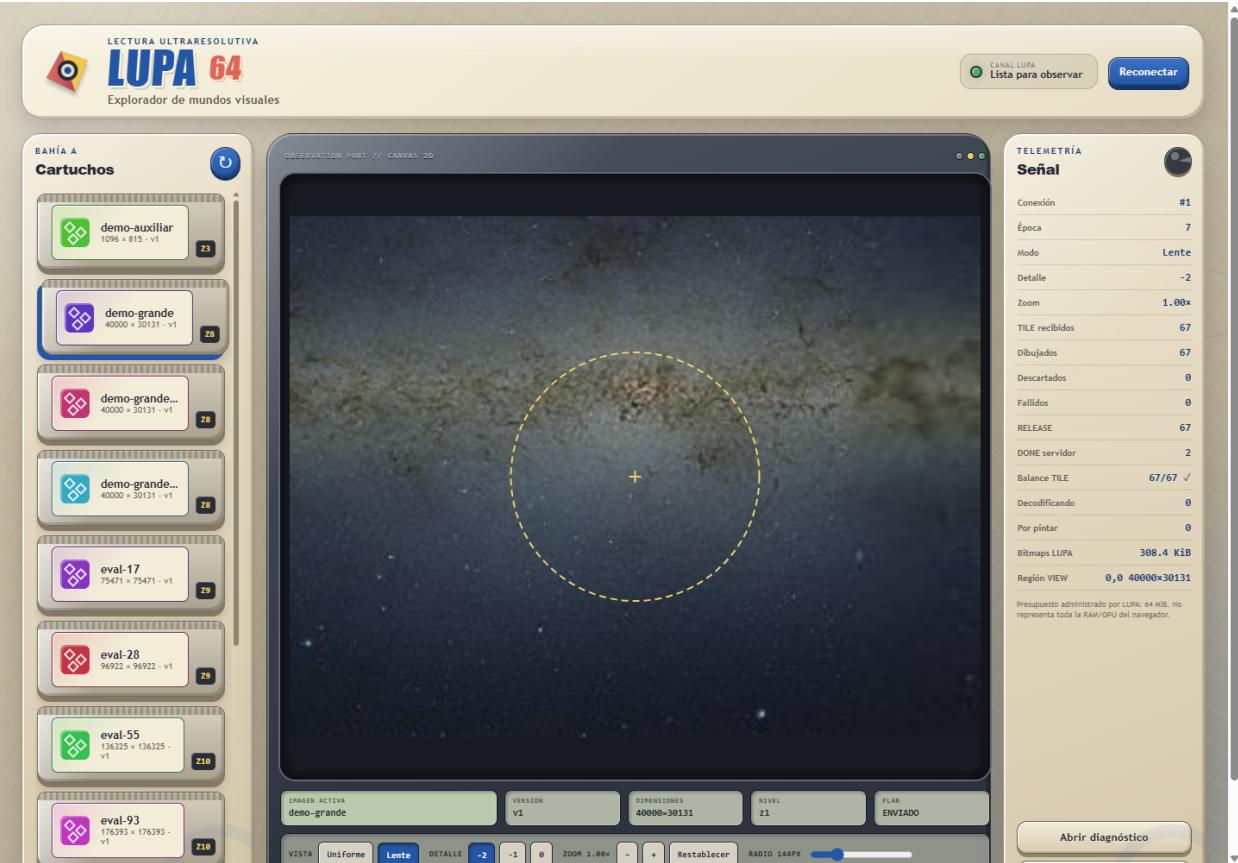


Detalle 0: PLAN z3. Conjunto de bitmaps: 4784104 bytes.



Detalle -2: PLAN z1. Quedan 369576 bytes; se cerraron 20 bitmaps. Reducir resolución elimina detalle almacenado, no solo cambia la escala CSS.

18. Foco, importación y protección



demo-grande/v1 en modo Lente. El foco usa un plan selectivo; la miniatura/contexto permanece disponible. Captura final real, sin reconstruir la imagen completa. Las capturas de eval-93 también se conservan en el paquete.

18.1 Importación aislada repetida

Caso	Evidencia y resultado
Entrada válida	prueba-real.jpg, 1096x815, 176062 bytes. a24-proof/v1: 29 teselas; 224916 bytes JPEG; import OK.
Entrada inválida	Archivo no imagen: salida 3. Hash del catálogo antes/después idéntico. No aparece a24-invalid.
Original conservado	SHA-256 antes/después idéntico: 783ab0e877d685b9a35a17de1c8e164dc179852d97bb738cf7b72499a6aa3459.
HTTP local	/api/catalog: 200. Solicitudes directas a originals, staging y una tesela por ruta de archivo: 404.

18.2 Confirmaciones y memoria

La reducción observó cierres reales de ImageBitmap, no solo un contador visual. La aceptación repetida validó identidad por imagen/versión/época y descarte tardío. Los tests backend repetidos cubren crédito idempotente y limpieza; las trazas no convierten ACK en una orden para liberar inmediatamente un bitmap todavía útil. Cada entrega se confirma terminalmente; su contenido puede permanecer en Canvas hasta invalidar la vista.